Mobile and Wireless Technologies — Fundamentals

Spring Semester 2026

WorkFlow

Field Team Coordination App

Project Documentation

Author 1: Adam Strelec
Author 2: Damián Styrak
Instructor:
Group: XY

February 24, 2026

Contents

1 Project Overview **3**
- 1.1 Purpose of the Application . . . 3
- 1.2 Technology Stack . . . 3
- 1.3 Mandatory Optional Requirements (PVP) . . . 3

2 User Roles and Use Cases **4**
- 2.1 User Roles . . . 4
- 2.2 Use Cases . . . 4
  - 2.2.1 Manager . . . 4
  - 2.2.2 Worker . . . 5

3 Functional Requirements **6**
- 3.1 Application Screens . . . 6
- 3.2 CRUD Operations . . . 6
- 3.3 Offline Mode . . . 6
- 3.4 WebSockets — Real-time Communication . . . 7
- 3.5 Push Notifications . . . 7
- 3.6 Permission Handling . . . 7
- 3.7 Other Mandatory Requirements . . . 7

4 System Architecture **8**
- 4.1 Architecture Overview . . . 8
- 4.2 Backend . . . 9
- 4.3 Mobile Application . . . 9

5 Data Model **10**
- 5.1 Entity-Relationship Diagram . . . 10
- 5.2 Entity Descriptions . . . 10
  - 5.2.1 users . . . 10
  - 5.2.2 jobs . . . 11
  - 5.2.3 job_assignments . . . 11
  - 5.2.4 job_photos . . . 11
  - 5.2.5 chat_messages . . . 12
  - 5.2.6 worker_locations . . . 12

6 Data Model **13**
- 6.1 Entity-Relationship Diagram . . . 13
- 6.2 Entity Descriptions . . . 13
  - 6.2.1 users . . . 13

6.2.2 jobs . . . 14
6.2.3 job_assignments . . . 14
6.2.4 job_photos . . . 14
6.2.5 chat_messages . . . 15
6.2.6 worker_locations . . . 15

7 Integrations **16**
7.1 Device Integrations . . . 16
7.1.1 Camera and Gallery . . . 16
7.1.2 GPS / Location . . . 16
7.1.3 Push Notifications . . . 16
7.2 Third-party Service Integrations . . . 16
7.2.1 PVP4 — Google SSO . . . 16
7.2.2 PVP5 — Firebase Analytics + Crashlytics . . . 17
7.2.3 PVP6 — Location Based Service . . . 17
7.2.4 React Native Maps — Google Maps . . . 17

8 Acceptance Tests (UAT) **18**
8.1 Positive Tests . . . 18
8.2 Negative Tests . . . 22

Chapter 1

Project Overview

1.1 Purpose of the Application

WorkFlow is a mobile application designed for coordinating field teams. It allows managers to create and assign work orders (jobs) to field workers, track their location in real time on a map, and communicate with them through a built-in chat. Workers receive push notifications about new assignments, can update job statuses, and upload photo documentation directly from the field.

The goal of the application is to replace inefficient email or phone-based coordination and give management an instant overview of job progress and team location.

1.2 Technology Stack

Layer	**Technology**
Frontend	React Native (cross-platform iOS / Android)
Backend	Node.js (Express)
Database	PostgreSQL
Real-time	WebSockets (Socket.IO)
Push notifications	Firebase Cloud Messaging (FCM)
Analytics	Firebase Analytics + Firebase Crashlytics
Authentication	JWT + Google SSO (OAuth 2.0)
Maps	React Native Maps (Google Maps)

Table 1.1: WorkFlow technology stack

1.3 Mandatory Optional Requirements (PVP)

In accordance with the project specification, the following three PVPs are implemented:

- **PVP4** — SSO authentication support (Google Sign-In)
- **PVP5** — Firebase services integration (Analytics + Crashlytics)
- **PVP6** — Location-based service (GPS worker tracking, geofencing around job sites)

Chapter 2

User Roles and Use Cases

2.1 User Roles

The application distinguishes between two main user roles:

Manager Creates, edits, and deletes jobs. Assigns jobs to workers. Tracks the team's location on the map in real time. Has access to an overview of all jobs and their statuses. Can communicate with workers via chat.

Worker Can only see jobs assigned to them. Updates the status of a job (e.g. *In Progress*, *Done*). Uploads photo documentation to a job. Shares their GPS location with the manager. Can communicate with the manager via chat.

2.2 Use Cases

2.2.1 Manager

#	Use case	Description
UC-M1	Login	Manager logs in via Google SSO or email and password.
UC-M2	Create job	Manager creates a new job with a title, description, GPS coordinates, and deadline.
UC-M3	Assign job	Manager assigns a job to one or more workers.
UC-M4	Edit / delete job	Manager edits or deletes an existing job.
UC-M5	Job overview	Manager sees a list of all jobs with their current statuses.
UC-M6	Team map	Manager sees live worker locations and job site pins on the map.
UC-M7	Chat	Manager communicates with workers via real-time chat.
UC-M8	Push notification	Manager receives a notification when a worker changes a job status to *Done*.

#	Use case	Description

Table 2.1: Use cases — Manager

2.2.2 Worker

#	Use case	Description
UC-W1	Login	Worker logs in via Google SSO or email and password.
UC-W2	My job list	Worker sees the list of jobs assigned to them.
UC-W3	Job detail	Worker opens a job's detail — description, map, photos, status.
UC-W4	Update status	Worker changes job status to *In Progress* or *Done*.
UC-W5	Upload photo	Worker takes a photo or picks one from the gallery and attaches it to the job.
UC-W6	Offline update	Worker updates job status without internet; change syncs after reconnection.
UC-W7	Chat	Worker communicates with the manager via real-time chat.
UC-W8	Push notification	Worker receives a notification about a newly assigned job.

Table 2.2: Use cases — Worker

Chapter 3

Functional Requirements

3.1 Application Screens

The application includes the following 7 functional screens (splash screen and login screen are not counted):

1. **Job List** — list of jobs (Manager: all jobs; Worker: assigned jobs only)
2. **Job Detail** — job detail view with status, description, photos, and status-change button
3. **Create / Edit Job** — form for creating or editing a job (Manager only)
4. **Map View** — map with live worker locations and job site pins
5. **Chat** — real-time conversation per job
6. **Worker List** — list of workers and their current status (Manager only)
7. **Profile / Settings** — user profile, logout, dark mode toggle

3.2 CRUD Operations

Entity	Operation	Description
Job	Create	Manager creates a new job via the form
Job	Read	Job list and detail view for both roles
Job	Update	Manager edits the job; Worker changes the status
Job	Delete	Manager deletes a job

Table 3.1: CRUD operations on the Job entity

3.3 Offline Mode

The application supports offline functionality using a local SQLite database (**WatermelonDB**). When the device has no internet connection:

- A Worker can change a job's status or add notes — the change is saved locally with a `pending_sync` flag.
- Once the connection is restored, changes are automatically synchronized with the server in the background.
- Conflicts (e.g. the job was deleted by the manager in the meantime) are resolved with a *server wins* policy: the server returns an error, the application notifies the user, and the local change is discarded.

3.4 WebSockets — Real-time Communication

The application uses a WebSocket connection (Socket.IO) for:

- **Live chat** — bidirectional messaging between Manager and Worker within a job.
- **Location sharing** — Worker periodically (every 10 s) sends their GPS position; Manager receives it in real time and sees it on the map.

3.5 Push Notifications

Trigger	Recipient	Behaviour
New job assigned (server)	Worker	Background notification; tap redirects to Job Detail
Job completed (app)	Manager	Background notification; tap redirects to Job Detail

Table 3.2: Push notifications

3.6 Permission Handling

The application requests the following system permissions:

- **Camera** — taking job photo documentation
- **Gallery / Files** — selecting an existing photo
- **Location (foreground + background)** — sharing the Worker's GPS position
- **Push notifications** — receiving FCM messages

3.7 Other Mandatory Requirements

- **Dark mode** — the application fully supports the system dark mode as well as a manual toggle in Settings.
- **User management** — persistent authentication via a JWT token stored in secure storage (`react-native-keychain`).

Chapter 4

System Architecture

4.1 Architecture Overview

The system consists of three main components:

1. **Mobile application (React Native)** — cross-platform client for iOS and Android
2. **Backend server (Node.js / Express)** — REST API + WebSocket server
3. **Database (PostgreSQL)** — persistent data store

Communication flows as follows:

- Mobile app ↔ Backend: **HTTPS REST API** for standard operations
- Mobile app ↔ Backend: **WebSocket (WSS)** for real-time chat and location sharing
- Backend → Mobile app: **Firebase Cloud Messaging** for push notifications
- Mobile app ↔ Firebase: **Firebase Analytics** + **Crashlytics** SDK

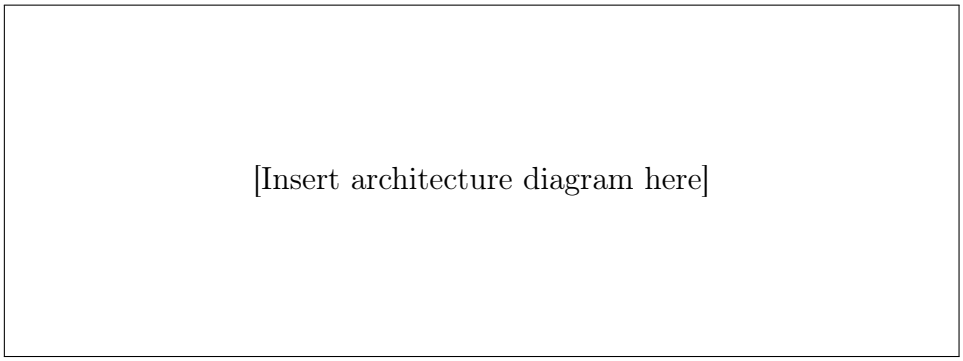


Figure 4.1: WorkFlow system architecture diagram

4.2 Backend

The backend is implemented in **Node.js** with the **Express** framework. It runs in a Docker container alongside a PostgreSQL database (docker-compose).

Main backend modules:

- `auth` — registration, login, Google OAuth, JWT issuance and verification
- `jobs` — CRUD endpoints for jobs, worker assignment
- `users` — user and role management
- `media` — binary content upload and download (photos)
- `socket` — Socket.IO handlers for chat and location
- `notifications` — FCM message dispatch

Interactive API testing is provided via **Swagger UI** at `/api-docs`.

4.3 Mobile Application

The application is built with **React Native** using a **Redux Toolkit** architecture:

- `api/` — axios HTTP client, WebSocket (Socket.IO client)
- `store/` — Redux slices and async thunks (Redux Toolkit)
- `db/` — local SQLite database via WatermelonDB
- `screens/` — React Native screens and navigation (React Navigation)
- `components/` — shared UI components

Chapter 5

Data Model

5.1 Entity-Relationship Diagram

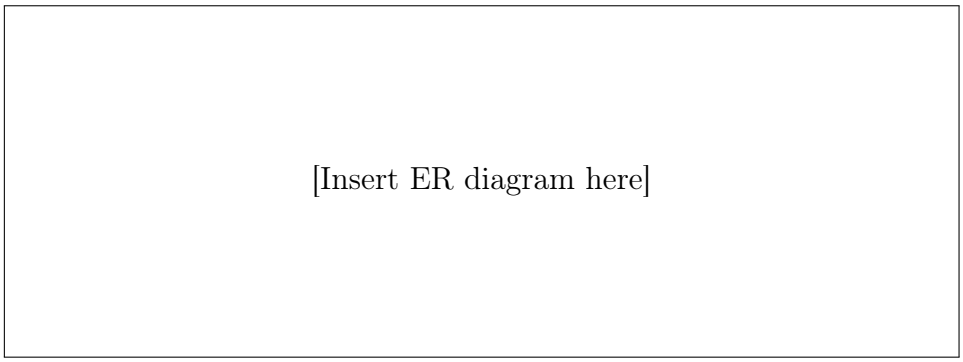


Figure 5.1: Entity-relationship diagram of the WorkFlow database

5.2 Entity Descriptions

5.2.1 users

Column	Type	Description
id	UUID (PK)	Primary key
email	VARCHAR(255)	Unique user email
password_hash	VARCHAR(255)	Bcrypt hash of password (null for SSO users)
name	VARCHAR(100)	Display name
role	ENUM	`manager` or `worker`
fcm_token	TEXT	Firebase token for push notifications
avatar_url	TEXT	URL of the profile picture
created_at	TIMESTAMP	Record creation time

Table 5.1: Table `users`

5.2.2 jobs

Column	Type	Description
id	UUID (PK)	Primary key
title	VARCHAR(200)	Job title
description	TEXT	Job description
status	ENUM	`pending`, `in_progress`, `done`
latitude	FLOAT	GPS latitude of the job site
longitude	FLOAT	GPS longitude of the job site
deadline	TIMESTAMP	Completion deadline
created_by	UUID (FK)	Reference to `users.id` (manager)
created_at	TIMESTAMP	Record creation time
updated_at	TIMESTAMP	Last update time

Table 5.2: Table `jobs`

5.2.3 job_assignments

Column	Type	Description
job_id	UUID (FK)	Reference to `jobs.id`
worker_id	UUID (FK)	Reference to `users.id`
assigned_at	TIMESTAMP	Time of assignment

Table 5.3: Table `job_assignments` (M:N jobs ↔ workers)

5.2.4 job_photos

Column	Type	Description
id	UUID (PK)	Primary key
job_id	UUID (FK)	Reference to `jobs.id`
uploaded_by	UUID (FK)	Reference to `users.id`
file_path	TEXT	Path to file on the server
mime_type	VARCHAR(50)	File type (e.g. `image/jpeg`)
created_at	TIMESTAMP	Upload time

Table 5.4: Table `job_photos`

5.2.5 chat_messages

Column	Type	Description
id	UUID (PK)	Primary key
job_id	UUID (FK)	Reference to `jobs.id`
sender_id	UUID (FK)	Reference to `users.id`
content	TEXT	Message text
sent_at	TIMESTAMP	Time sent

Table 5.5: Table `chat_messages`

5.2.6 worker_locations

Column	Type	Description
worker_id	UUID (FK)	Reference to `users.id`
latitude	FLOAT	Current latitude
longitude	FLOAT	Current longitude
recorded_at	TIMESTAMP	Time of the location record

Table 5.6: Table `worker_locations` (latest position per worker)

Chapter 6

Data Model

6.1 Entity-Relationship Diagram

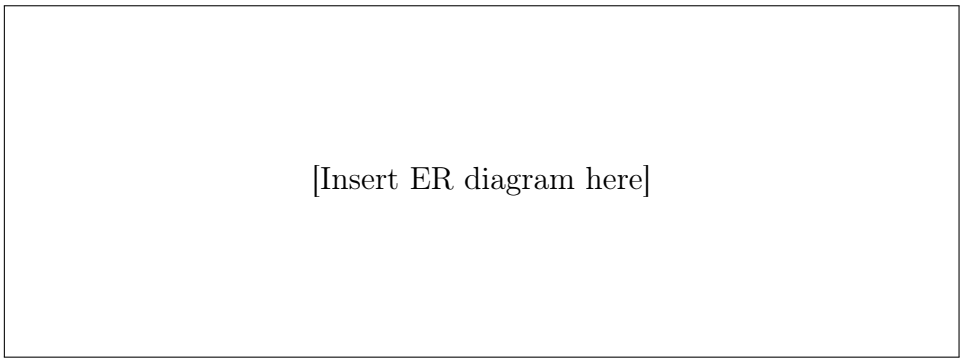


Figure 6.1: Entity-relationship diagram of the WorkFlow database

6.2 Entity Descriptions

6.2.1 users

Column	Type	Description
id	UUID (PK)	Primary key
email	VARCHAR(255)	Unique user email
password_hash	VARCHAR(255)	Bcrypt hash of password (null for SSO users)
name	VARCHAR(100)	Display name
role	ENUM	`manager` or `worker`
fcm_token	TEXT	Firebase token for push notifications
avatar_url	TEXT	URL of the profile picture
created_at	TIMESTAMP	Record creation time

Table 6.1: Table `users`

6.2.2 jobs

Column	Type	Description
id	UUID (PK)	Primary key
title	VARCHAR(200)	Job title
description	TEXT	Job description
status	ENUM	`pending`, `in_progress`, `done`
latitude	FLOAT	GPS latitude of the job site
longitude	FLOAT	GPS longitude of the job site
deadline	TIMESTAMP	Completion deadline
created_by	UUID (FK)	Reference to `users.id` (manager)
created_at	TIMESTAMP	Record creation time
updated_at	TIMESTAMP	Last update time

Table 6.2: Table `jobs`

6.2.3 job_assignments

Column	Type	Description
job_id	UUID (FK)	Reference to `jobs.id`
worker_id	UUID (FK)	Reference to `users.id`
assigned_at	TIMESTAMP	Time of assignment

Table 6.3: Table `job_assignments` (M:N jobs ↔ workers)

6.2.4 job_photos

Column	Type	Description
id	UUID (PK)	Primary key
job_id	UUID (FK)	Reference to `jobs.id`
uploaded_by	UUID (FK)	Reference to `users.id`
file_path	TEXT	Path to file on the server
mime_type	VARCHAR(50)	File type (e.g. `image/jpeg`)
created_at	TIMESTAMP	Upload time

Table 6.4: Table `job_photos`

6.2.5 chat_messages

Column	Type	Description
id	UUID (PK)	Primary key
job_id	UUID (FK)	Reference to `jobs.id`
sender_id	UUID (FK)	Reference to `users.id`
content	TEXT	Message text
sent_at	TIMESTAMP	Time sent

Table 6.5: Table `chat_messages`

6.2.6 worker_locations

Column	Type	Description
worker_id	UUID (FK)	Reference to `users.id`
latitude	FLOAT	Current latitude
longitude	FLOAT	Current longitude
recorded_at	TIMESTAMP	Time of the location record

Table 6.6: Table `worker_locations` (latest position per worker)

Chapter 7

Integrations

7.1 Device Integrations

7.1.1 Camera and Gallery

The application uses the `react-native-image-picker` library to access the camera and photo gallery. Before first use, the app requests the necessary permissions (`CAMERA`, `READ_MEDIA_IMAGES` on Android; `NSCameraUsageDescription`, `NSPhotoLibraryUsageDescription` on iOS). Photos are sent to the server as `multipart/form-data` without Base64 encoding.

7.1.2 GPS / Location

The `react-native-geolocation-service` library provides access to the device's location. While actively working, a Worker shares their position every 10 seconds via WebSocket. The app requests `ACCESS_FINE_LOCATION` (Android) and `NSLocationWhenInUseUsageDescri` / `NSLocationAlwaysUsageDescription` (iOS). The map (`react-native-maps` with Google Maps provider) displays live worker markers and static job site pins.

7.1.3 Push Notifications

Push notifications are implemented using **Firebase Cloud Messaging** via the `@react-native-fire` library. The device's FCM token is stored on the backend (column `users.fcm_token`) and updated on every login. The new-job notification (sent by the server) includes a `data` payload with `job_id`, enabling a direct redirect to Job Detail on tap.

7.2 Third-party Service Integrations

7.2.1 PVP4 — Google SSO

Google authentication is implemented using `@react-native-google-signin/google-signin` and Google OAuth 2.0. After a successful Google Sign-In, the app sends the `id_token` to the backend endpoint `POST /api/auth/google`, where it is verified and a JWT is issued.

7.2.2 PVP5 — Firebase Analytics + Crashlytics

- **Firebase Analytics** (`@react-native-firebase/analytics`) — tracks key events: `job_created`, `job_status_changed`, `photo_uploaded`, `chat_message_sent`. Analytics output is included as an appendix to the documentation.
- **Firebase Crashlytics** (`@react-native-firebase/crashlytics`) — automatically captures application crashes and non-fatal errors. A crash report from the testing phase is included in the documentation appendix.

7.2.3 PVP6 — Location Based Service

In addition to live location tracking, **geofencing** is implemented: when a Worker moves more than 500 m away from the GPS coordinates of their assigned job, the app displays an alert. Geofencing is handled by `react-native-geofencing`.

7.2.4 React Native Maps — Google Maps

The map is rendered using `react-native-maps` with the Google Maps provider. It displays:

- colour-coded markers for each active Worker's position (updated via WebSocket),
- pins for each job site, colour-coded by status (`pending` = grey, `in_progress` = orange, `done` = green).

Chapter 8

Acceptance Tests (UAT)

Each test consists of at least 7 steps, spans at least two screens, and forms a logical, self-contained scenario. Account creation and login are not included as valid UAT scenarios.

8.1 Positive Tests

UAT-01: Create a New Job

Preconditions	A user with the Manager role is logged in. The app has an internet connection.
Postconditions	The new job is saved to the database and appears in the job list.
Steps:	
1.	User is on the **Job List** screen.
2.	Taps the + button (Create Job).
3.	The **Create Job** screen appears.
4.	Fills in the title, description, and selects a deadline.
5.	Taps on the map to set the job location.
6.	Taps the **Save** button.
7.	App navigates back to **Job List**, where the new job appears at the top.
Result	**PASS** / **FAIL**

Table 8.1: UAT-01: Create a new job

UAT-02: Assign a Worker to a Job

Preconditions	Manager is logged in. At least one Worker and one job exist in the system.
Postconditions	Worker is assigned to the job; Worker receives a push notification.
Steps:	
1.	Manager opens **Job List** and taps an existing job.
2.	**Job Detail** screen is displayed.
3.	Taps **Assign Worker**.
4.	A list of workers appears; Manager selects one.
5.	Confirms the selection with the **Confirm** button.
6.	The assigned worker's name appears in Job Detail.
7.	The Worker's device receives a push notification: *"You have been assigned a new job"*.
Result	**PASS** / **FAIL**

Table 8.2: UAT-02: Assign a worker to a job

UAT-03: Worker Updates a Job Status

Preconditions	Worker is logged in and has an assigned job with status *Pending*. App has internet.
Postconditions	Job status changes to *In Progress*; Manager sees the update.
Steps:	
1.	Worker opens **Job List** (My Jobs).
2.	Taps a job with status *Pending*.
3.	**Job Detail** screen is displayed.
4.	Taps the **Start Job** button.
5.	A confirmation dialog appears; Worker taps **Confirm**.
6.	Job status changes to *In Progress* with an appropriate badge.
7.	On the Manager's device, the job status updates in real time in **Job List**.
Result	**PASS** / **FAIL**

Table 8.3: UAT-03: Worker updates a job status

UAT-04: Upload a Photo to a Job

Preconditions	Worker is logged in, has an assigned job. App has internet and camera permission.
Postconditions	Photo is uploaded to the server and visible in Job Detail.
Steps:	
1.	Worker opens **Job Detail** of an assigned job.
2.	Scrolls to the *Photo Documentation* section and taps +.
3.	A picker appears: *Camera* or *Gallery*; Worker selects *Camera*.
4.	The camera opens; Worker takes a photo and confirms.
5.	App uploads the photo to the server.
6.	A progress indicator is shown during the upload.
7.	After a successful upload, the new photo appears in the job's gallery.
Result	**PASS** / **FAIL**

Table 8.4: UAT-04: Upload a photo to a job

UAT-05: Real-time Chat Between Manager and Worker

Preconditions	Both Manager and Worker are logged in and have the chat screen of the same job open.
Postconditions	Messages appear on both devices in real time.
Steps:	
1.	Manager opens **Job Detail** and taps **Chat**.
2.	Worker opens the same job and taps **Chat**.
3.	Manager types a message and taps **Send**.
4.	The message appears instantly on the Worker's device without refreshing.
5.	Worker replies with a message and taps **Send**.
6.	The message appears instantly on the Manager's device.
7.	Both devices show the full message history in the correct chronological order.
Result	**PASS** / **FAIL**

Table 8.5: UAT-05: Real-time chat

UAT-06: Track Worker Location on the Map

Preconditions	Manager is logged in on the **Map View** screen. Worker is logged in with location sharing enabled.
Postconditions	Manager sees the Worker's current location on the map.
Steps:	
1.	Manager opens **Map View**.
2.	Job site pins are visible on the map.
3.	Worker moves physically (or the simulator changes the location).
4.	Worker opens the app — it begins sending GPS position.
5.	A Worker marker appears on the Manager's map within 15 seconds.
6.	After further movement, the marker shifts to the new position.
7.	Manager taps the Worker's marker — the Worker's name and status are shown.
Result	**PASS** / **FAIL**

Table 8.6: UAT-06: Track worker location on the map

UAT-07: Offline Job Status Update

Preconditions	Worker is logged in with an assigned job. Device has no internet (airplane mode on).
Postconditions	After internet is restored, the status change is synced to the server.
Steps:	
1.	Worker enables airplane mode on the device.
2.	Opens **Job List** — jobs are still visible (local cache).
3.	Opens **Job Detail** and taps **Mark as Done**.
4.	App shows status *Done* with an offline sync icon (clock).
5.	Worker disables airplane mode.
6.	App detects connectivity and triggers background sync.
7.	The clock icon disappears; the job status in **Job List** is updated on the server.
Result	**PASS** / **FAIL**

Table 8.7: UAT-07: Offline job status update

8.2 Negative Tests

UAT-08: Worker Attempts to Create a Job

Preconditions	A user with the Worker role is logged in.
Postconditions	App does not allow job creation; server returns a 403 error.
Steps:	
1.	Worker opens **Job List**.
2.	The + (Create Job) button is not visible (hidden at the UI level).
3.	An attacker attempts to call `POST /api/jobs` directly using the Worker's JWT.
4.	The server verifies the user's role.
5.	Server returns `403 Forbidden`.
6.	App displays an error message: *"You do not have permission for this action."*.
7.	No new job is created in the database.
Result	**PASS** / **FAIL**

Table 8.8: UAT-08: Unauthorised attempt to create a job (Worker)

UAT-09: Sending a Chat Message Without Internet

Preconditions	User is logged in. Device has no internet (airplane mode on).
Postconditions	Message is not sent; app informs the user of the connection error.
Steps:	
1.	User enables airplane mode.
2.	Opens **Chat** for a job.
3.	Types a message and taps **Send**.
4.	App attempts to establish a WebSocket connection.
5.	Connection fails — app shows a banner: *"You are offline. Chat is unavailable."*.
6.	Message is not sent and is not saved to local cache.
7.	After internet is restored, the banner disappears and chat works normally.
Result	**PASS** / **FAIL**

Table 8.9: UAT-09: Attempt to chat without internet connection

UAT-10: Delete a Job with an Active Worker

Preconditions	Manager is logged in. A job with status *In Progress* and an assigned Worker exists.
Postconditions	App shows a warning; after confirmation the job is deleted and Worker is notified.

Steps:

1.	Manager opens **Job Detail** of an *In Progress* job.
2.	Taps **Delete Job**.
3.	A warning dialog appears: *"This job has an assigned worker. Are you sure you want to delete it?"*.
4.	Manager confirms the deletion.
5.	Server deletes the job and its assignments from the database.
6.	Manager is redirected to **Job List** where the job is gone.
7.	Worker receives a push notification: *"A job was deleted by the Manager"* and it disappears from their list.

Result **PASS** / **FAIL**

Table 8.10: UAT-10: Delete an active job